Merge Sorted Arrays

AlgoExpert · Very Hard · Heaps

The problem

- Merge k sorted lists into one sorted list using a min-heap of (value, list, index).

Intuition

- A min-heap holds the head of each list; popping it always gives the next smallest overall.

Brute force vs optimal

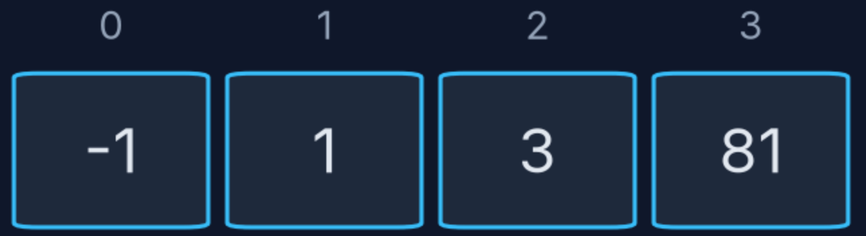
- Concatenate and sort → O(N log N)
- Repeated two-way merges → O(N·k)
- Min-heap of k heads → O(N log k)

Approach

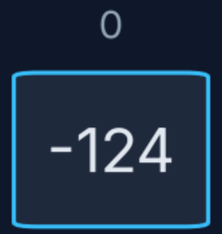
- 1. Push each list's first element
- 2. Pop the min and append
- 3. Push the next from that list
- 4. Repeat

Worked example

heap (one per array)



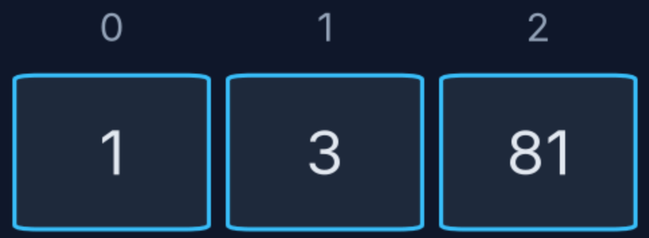
output



Pop -124 from array 2

Worked example

heap (one per array)



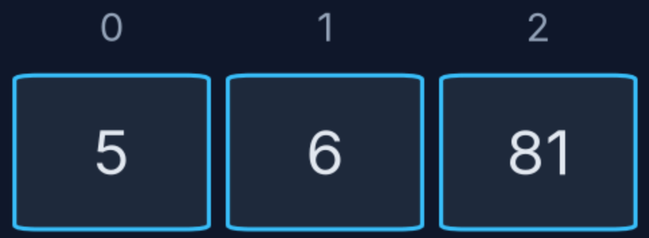
output



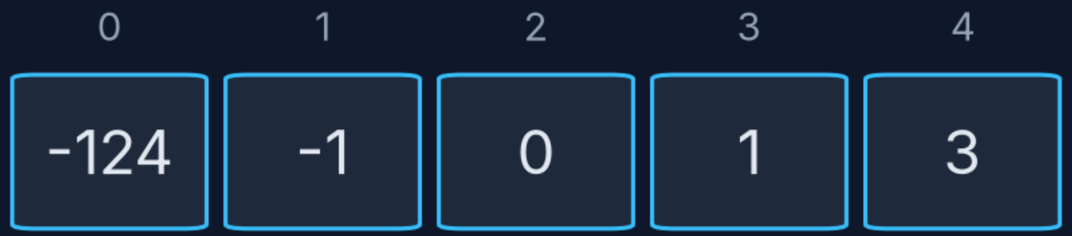
Pop 0 from array 1

Worked example

heap (one per array)



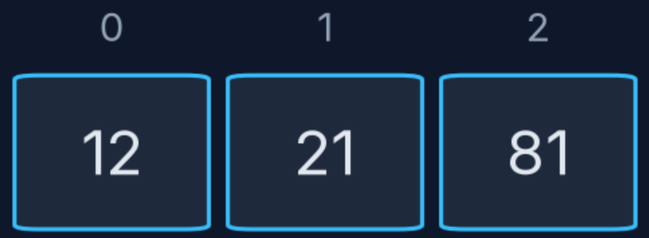
output



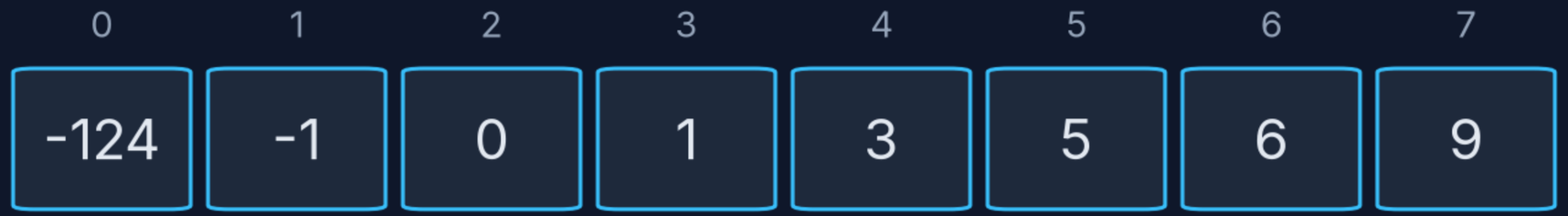
Pop 3 from array 3

Worked example

heap (one per array)



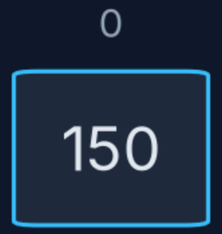
output



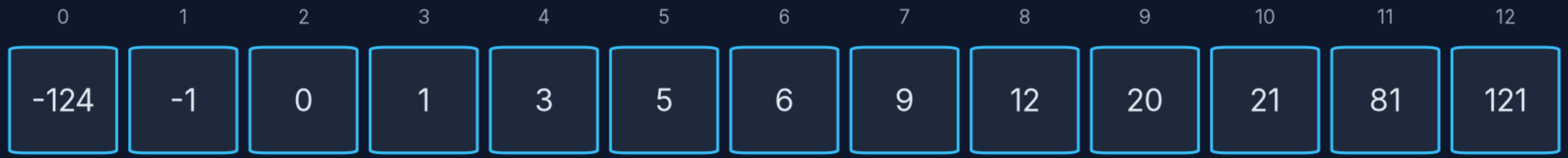
Pop 9 from array 0

Worked example

heap (one per array)



output



Pop 121 from array 2

Java solution

```java
public class MergeSortedArrays {
    public static List<Integer> mergeSortedArrays(List<List<Integer>> arrays) {
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> Integer.compare(a[0], b[0]));
        for (int i = 0; i < arrays.size(); i++) if (!arrays.get(i).isEmpty()) pq.add(new int[]{arrays.get(i).get(0), i, 0});
        List<Integer> out = new ArrayList<>();
        while (!pq.isEmpty()) {
            int[] t = pq.poll();
            out.add(t[0]);
            List<Integer> src = arrays.get(t[1]);
            if (t[2] + 1 < src.size()) pq.add(new int[]{src.get(t[2] + 1), t[1], t[2] + 1});
        }
        return out;
    }
}
```

Complexity

- Time: O(n log k)
- Space: O(n + k)

Edge cases & pitfalls

- Empty arrays
- Store (value, array, index)

Interview tips & follow-ups

- Mention divide-and-conquer merging

Recap

- A min-heap holds the head of each list; popping it always gives the next smallest overall.
- Complexity: O(n log k)